MULTICAST-AWARE LARGE NEIGHBORHOOD SEARCH

for Service Function Chain Embedding in NFV-Enabled Networks

K Gowri Mahesh
Department of Computer Science and Engineering
National Institute of Technology Warangal
Academic Year 2026–2027

Based on Asgarian, Mirjalily and Luo (2021) and the project's latest validation results

Abstract

Network Function Virtualization (NFV) runs network services such as firewalls and load balancers as software on general-purpose servers. A Service Function Chain (SFC) requires traffic to pass through a specified order of virtual network functions (VNFs). In multicast SFC embedding, one source sends traffic to several destinations, so the optimizer must decide both where to place VNFs and how to route traffic while allowing destinations to share processing and network links. The decisions are coupled: sharing fewer VNF instances can increase routing cost, while separate instances can consume more compute resources.

This report studies a multicast-aware large-neighborhood search method (MA-ALNS) that starts from an LP-based sequential heuristic and revisits placements using problem-specific destroy and repair operators. The main operators target excessive replication, poor sharing, and highly utilized links; a sharing-aware repair attempts to consolidate compatible VNF assignments. On the reported Palmetto-topology experiments, MA-ALNS lowers mean cost on some larger destination sets, mostly by reducing VNF instances. At 40 destinations, mean total cost decreases from 21.365 to 18.860 (11.72%), and mean VNF instances decrease from 17.0 to 14.5 (14.71%). However, paired Wilcoxon tests do not reach the 0.05 significance threshold, and runtime grows from roughly 0.13 seconds at four destinations to 6.75 seconds at 40. These results support an instance-dependent improvement mechanism, not a universal guarantee of superiority.

Keywords: NFV; multicast; service function chain; VNF placement; multicast routing; large neighborhood search.

1. The problem and why it is hard

1.1 What must be decided?

Represent the physical network as a directed graph G = (V, E). Nodes are servers or routers; directed edges are communication links. A subset V_F of nodes can host VNFs. Each eligible node has limited computing capacity, and each edge has limited bandwidth. A multicast request specifies a source s, destination set D, traffic rate λ, and ordered service chain F = (f_1, f_2, …, f_m).

The optimizer must choose where each VNF is deployed, which destinations share each instance, and which links carry the resulting multicast traffic. Every destination must pass through f_1, then f_2, and so on before reaching its endpoint. A short route that skips a required function is invalid.

1.2 The sharing tradeoff

In unicast routing, each request has one destination. In multicast, routes may share a prefix and branch later. Sharing can reduce the number of deployed VNFs and avoid sending duplicate traffic on the same shared links. However, forcing all destinations through one shared VNF may create a detour. The optimizer must balance network traffic cost against VNF-instance cost while preserving the SFC order and resource limits.

Illustrative configuration	VNF instances	Used links	Objective at λ = 0.1
Separate downstream placement	3	5	3 + 5 × 0.1 = 3.50
More shared placement	2	6	2 + 6 × 0.1 = 2.60

The example from the source paper illustrates that fewer VNF instances can justify additional links under the stated objective. It is an illustration of the tradeoff, not proof that either configuration is globally optimal for every topology.

1.3 Objective and feasibility

The original objective is the total traffic carried over links plus the number of deployed VNF instances. In compact notation:

$$C = L + N_VNF, \quad L = \Sigma_(i,j)\in E\ y_ij$$

Here y_{ij} is the aggregate traffic rate on link (i,j), and N_VNF is the deployed-instance count. A feasible solution must satisfy all of the following:

- Node capacity: the compute demand on each host does not exceed its capacity.
- Link capacity: the aggregate traffic on every link does not exceed its bandwidth capacity.
- SFC order: each destination traverses all required functions in order.

- Connectivity and flow conservation: traffic reaches every requested destination.
- Valid placement: each VNF is deployed only on an NFV-enabled node.

Feasibility checks establish that these constraints are met; they do not prove global optimality.

2. Previous methods and their limitations

The reference paper [1] formulates multicast SFC embedding as a mixed-binary optimization problem and introduces two solution methods. Related work also studies multicast topology construction and service-function trees [2–4].

Method	How it works	Strength	Relevant limitation
Exact MILP / branch-and-bound	Searches a discrete placement-and-routing model.	Can certify an optimum if solved to completion.	Worst-case computation rises sharply on NP-hard instances.
MPSUM-R [1]	Penalty-based relaxation, successive upper-bound minimization, rounding, then routing.	More scalable than an exact integer solve in the paper's tests.	Requires optimization machinery; rounding can affect placement quality.
Two-stage heuristic [1]	Sequential LP-based placement and rounding, followed by routing.	Very fast; the paper reports near-optimal average cost in its tests.	The staged decisions can limit later placement/routing reconsideration.
Tree / Steiner methods [2,3]	Build a shared multicast topology, possibly with VNF placement.	Explicitly exploits shared branches.	Function ordering and resource capacities make the joint problem difficult.
Generic LNS / ALNS [5]	Destroy part of a solution and repair it.	Can escape some poor incumbent solutions.	Generic random operators may not target multicast sharing structure.

Baseline fidelity: the project's PaperHeuristic follows the paper's Stage-1 LP placement and rounding logic. Stage 2 uses a shortest-path-based routing approximation rather than reproducing the paper's complete routing optimization. It is therefore a paper-inspired baseline, not an exact reproduction of every algorithmic detail.

3. Proposed method: MA-ALNS

MA-ALNS begins with a feasible solution from the LP-based sequential heuristic. At each iteration, a destroy operator removes selected placements or assignments; a repair operator reconstructs them. The candidate is checked for feasibility and evaluated with the original objective. The algorithm retains the best feasible solution found and uses a simulated-annealing-style acceptance rule to allow exploration of some temporarily worse candidates.

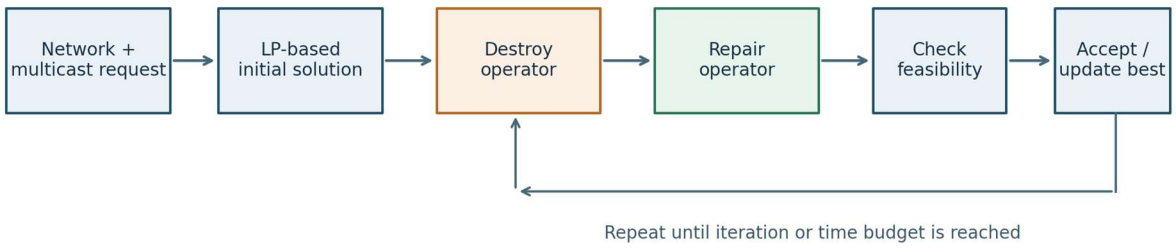


Figure 1. MA-ALNS workflow.

3.1 Destroy operators

- **High-replication destroy.** Targets a function replicated across many nodes or destination assignments. Repair then searches for a smaller set of shared instances.
- **Poor-sharing destroy.** Removes assignments that appear to miss feasible sharing opportunities so they can be rebuilt together.

- **Bottleneck-aware destroy.** Uses link utilization $u_{ij} = load_{ij} / capacity_{ij}$ to target placements associated with highly used links. Its benefit must be measured; the operator cannot be assumed to improve every instance.

3.2 Repair operators

- **Greedy repair.** Chooses a feasible placement with the smallest immediate incremental score.
- **Regret-2 repair.** Computes the best and second-best feasible insertion costs; their difference is regret. High-regret assignments are handled first. The implementation was corrected to handle zero or one feasible candidate.
- **Sharing-aware repair.** Evaluates candidates across multiple destinations and favours feasible reuse of an existing VNF when it reduces the overall objective.

3.3 Algorithm outline

1. Construct an initial feasible solution using the LP-based sequential heuristic.
2. Choose a destroy operator and remove selected assignments or placements.
3. Choose a repair operator and rebuild the affected portion of the solution.
4. Check SFC ordering, connectivity, node capacity, and link capacity.
5. Evaluate total cost; apply the acceptance rule and update the best solution.
6. Repeat until the iteration or runtime budget is reached.

The method is heuristic: no global-optimality proof or approximation guarantee has been established. The evidence is empirical and based on feasibility checks, illustrative-example validation, and comparative experiments.

4. Experimental setup

The experiments use the Palmetto topology referenced in [1] and generated multicast sessions. The paper-style settings are traffic rate $\lambda = 0.1$, link capacity $C_{ij} = 5$, node compute capacity $\mu_i = 20$, and VNF processing requirement $\omega_f = 1$. Destination scaling varies $|D|$ over 4, 8, 12, 16, 20, 30, and 40 with 10 functions. Function scaling varies the number of functions over 4, 8, 12, 16, and 20 with 10 destinations. The reported paired comparisons use 20 matched seeds.

Metric	Meaning
Total cost	Link cost + number of deployed VNF instances.
Link cost	Aggregate traffic carried over links; not just the number of distinct links.
VNF instances	Total deployed function instances.
Runtime	Wall-clock algorithm execution time.
Feasibility	Whether the checked capacity, connectivity and SFC-order constraints hold.

The destination-scaling, function-scaling, and ablation experiments use separately generated configurations. Their means should be interpreted within their own experiment set, not mixed as if they were repeated measurements of one identical 12-destination instance.

5. Results

5.1 Destination scaling

Destinations	Paper cost	MA-ALNS cost	Cost drop	VNFs: paper → MA	MA time (s)	Runtime ×
4	11.465	11.465	0.00%	10.00 → 10.00	0.129	8.0×
8	12.160	12.160	0.00%	10.00 → 10.00	0.330	17.4×
12	13.940	12.910	7.39%	11.05 → 10.00	0.697	31.3×
16	16.510	14.520	12.05%	13.50 → 11.50	1.072	40.7×
20	17.435	16.920	2.95%	14.00 → 13.50	1.771	55.6×
30	19.470	18.435	5.32%	15.50 → 14.50	3.940	67.0×

Destinations	Paper cost	MA-ALNS cost	Cost drop	VNFs: paper → MA	MA time (s)	Runtime ×
40	21.365	18.860	11.72%	17.00 → 14.50	6.754	80.3×

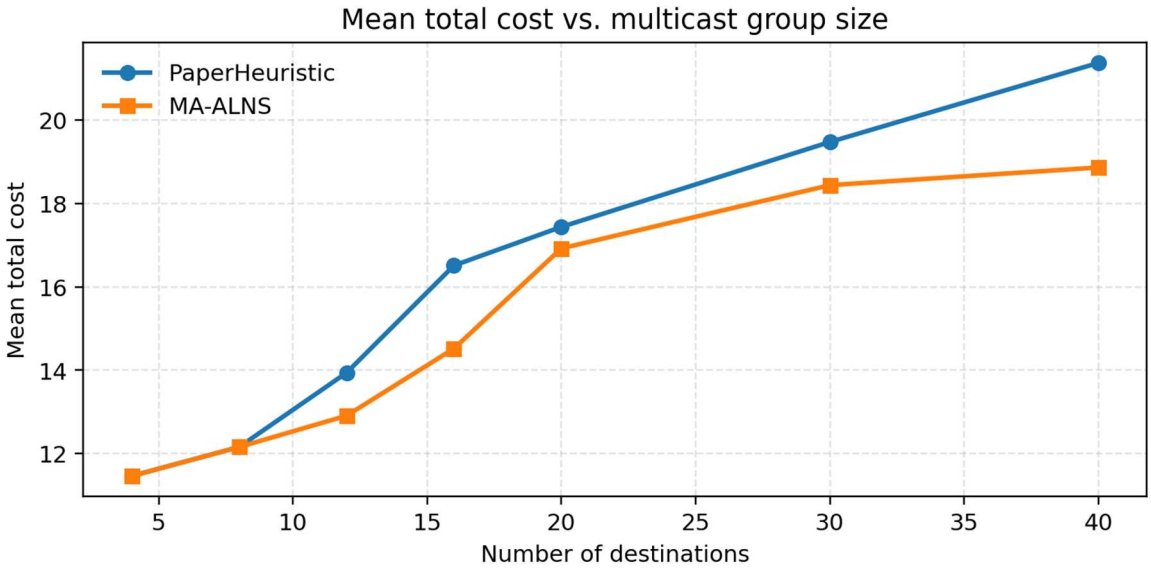


Figure 2. Mean total cost versus destination count.

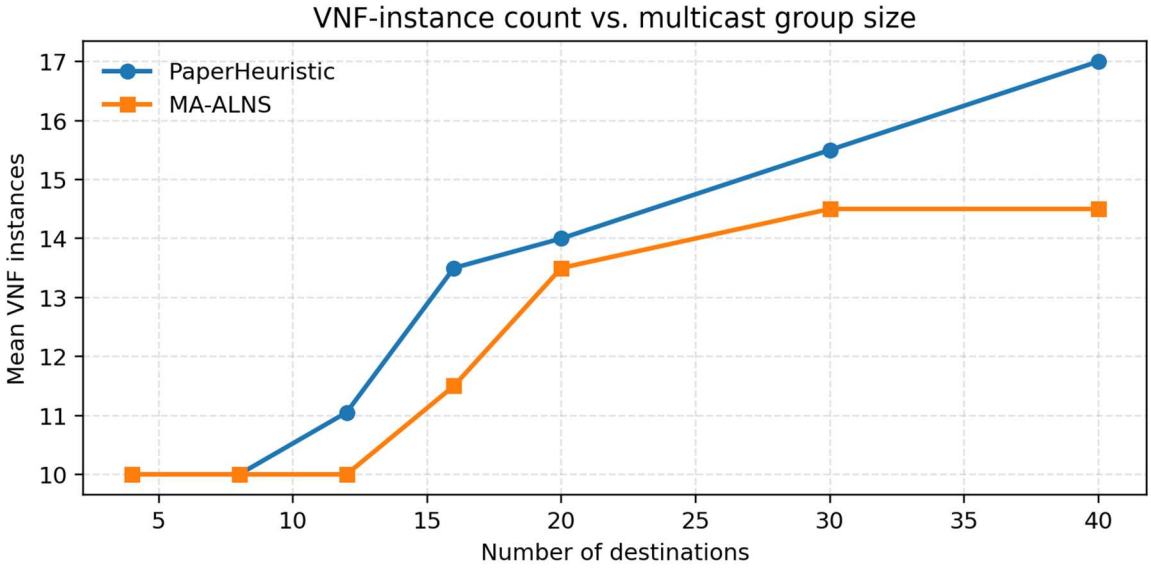


Figure 3. Mean number of VNF instances versus destination count.

At 40 destinations, total cost falls from 21.365 to 18.860: a reduction of 2.505, or 11.72%. Mean VNF count falls from 17.0 to 14.5, a reduction of 14.71%. Link cost changes only from 4.365 to 4.360, so the saving is almost entirely due to VNF consolidation. At 12 and 16 destinations, link cost is slightly higher under MA-ALNS, but the reduction in VNF count more than offsets that increase.

The trend is not monotonic: the 20-destination improvement is smaller than the 16-destination improvement. The data support instance-dependent gains on some larger multicast requests, not a guarantee that more destinations always yield a larger percentage improvement.

5.2 Statistical tests

Destinations	Improved seeds	Tied seeds	Worsened seeds	Wilcoxon p

Destinations	Improved seeds	Tied seeds	Worsened seeds	Wilcoxon p
4	0	20	0	N/A (all tied)
8	0	20	0	N/A (all tied)
12	3	17	0	0.2507
16	4	16	0	0.1411
20	4	16	0	0.1411
30	5	15	0	0.0770
40	5	15	0	0.0770

No reported paired Wilcoxon test reaches $p < 0.05$. Many matched seeds are exact ties; only 3–5 seeds improve at destination counts from 12 to 40. The mean cost reductions are therefore promising but statistically inconclusive under the reported tests. They must not be described as statistically significant.

5.3 Function scaling

Functions	Paper cost	MA-ALNS cost	Observation
4	6.665	6.460	Small gain
8	10.950	10.560	Small gain
12	15.060	15.060	Tie
16	18.395	18.395	Tie
20	23.515	23.515	Tie

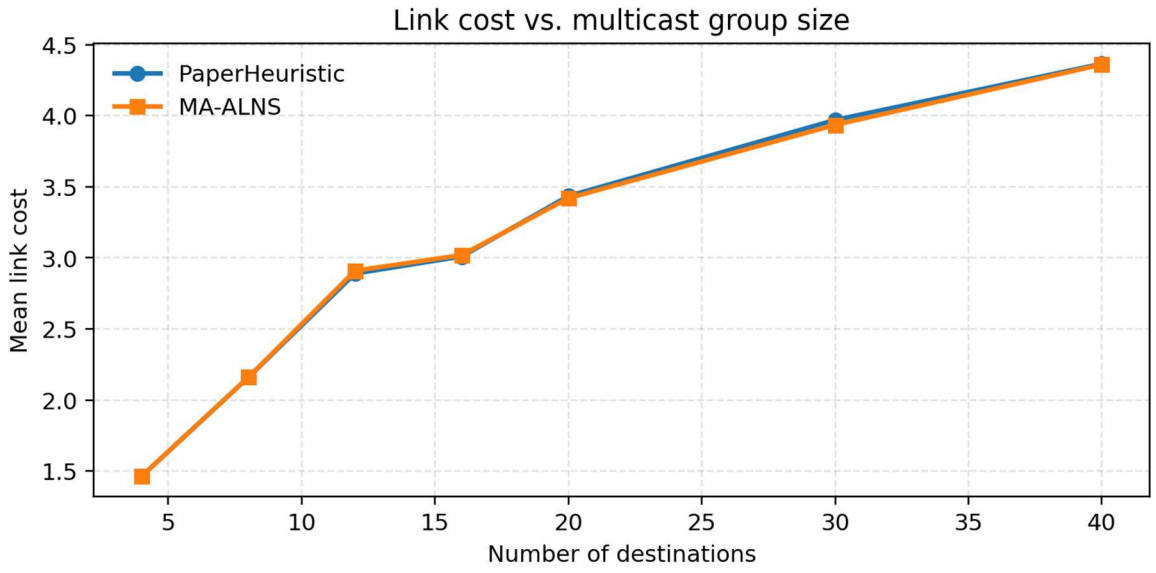


Figure 4. Link cost changes little compared with VNF-instance count.

For 12, 16, and 20 functions, the reported costs tie for all listed seeds. In those generated chain instances, the baseline appears to have little placement inefficiency for MA-ALNS to exploit. This is limited evidence for this workload, not a universal claim about deep SFCs.

5.4 Ablation and operator value

Method	Mean cost	Link cost	Mean VNFs	Mean time (s)

Method	Mean cost	Link cost	Mean VNFs	Mean time (s)
PaperHeuristic	14.645	2.645	12.0	0.0196
Standard ALNS	14.645	2.645	12.0	0.0712
Multicast-aware ALNS	14.145	2.645	11.5	0.6353
Adaptive variant	14.145	2.645	11.5	0.6210

In this separate 12-destination, 10-function ablation, standard ALNS does not improve the baseline. The multicast-aware variant lowers mean cost by 0.500, entirely through a reduction of 0.5 VNF instances in the mean. Adaptive operator weighting gives the same reported mean cost, and broader results do not show a consistent benefit from adaptation.

Link capacity	With bottleneck	Without bottleneck	PaperHeuristic	Feasibility
0.25	12.910	13.865	13.940	1.0
0.5	12.910	13.865	13.940	1.0
1.0	12.910	13.865	13.940	1.0
5.0	12.910	13.865	13.940	1.0

The reported bottleneck ablation gives a cost of 12.910 with the operator and 13.865 without it, a descriptive decrease of 0.955 (about 6.9%). Yet costs are identical across the four capacity values and feasibility is 1.0 throughout. This supports a difference between the tested operator configurations, but does not establish that the benefit increases with congestion or prevents infeasibility.

5.5 Runtime

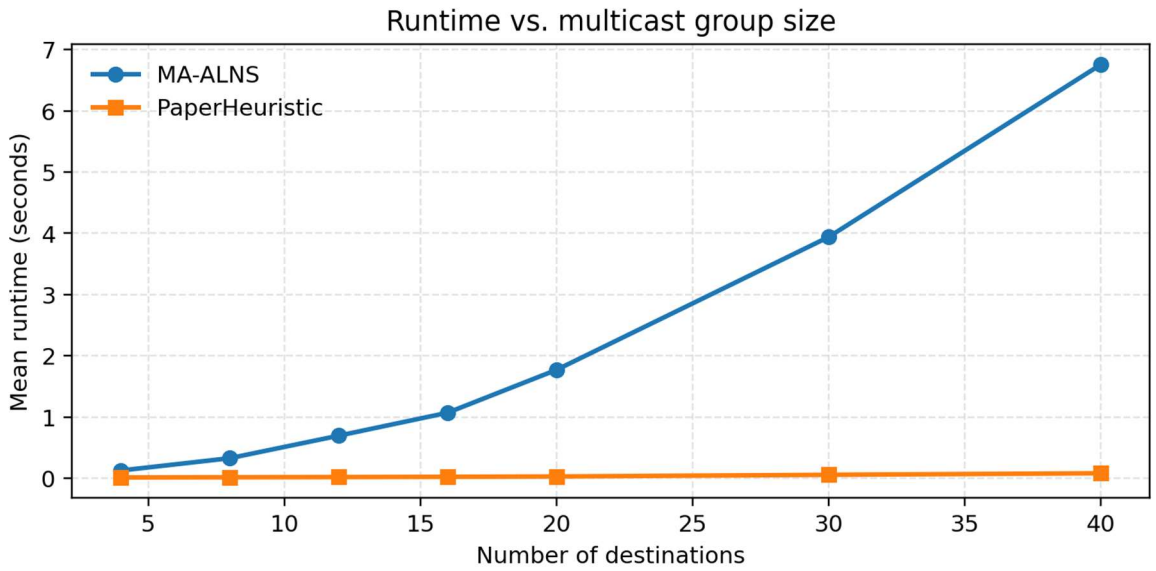


Figure 5. Runtime grows rapidly with destination count.

Mean runtime increases from approximately 0.129 seconds at four destinations to 6.754 seconds at 40 destinations. The reported runtime multiplier rises from 8.0× to 80.3×. This is the main practical weakness. A few seconds may be acceptable for some offline planning workloads, but the current implementation is costly for frequent real-time re-embedding.

6. Interpretation: what the evidence establishes

- Problem-specific neighborhoods matter in the reported ablation: generic ALNS matches the sequential baseline, while multicast-aware operators find lower-cost solutions.

- VNF consolidation is the dominant mechanism at larger destination counts. At 40 destinations, VNF count falls by 14.71%, while link cost falls by only about 0.11%.
- The benefit is instance-dependent and not statistically significant at the 0.05 level in the reported paired tests.
- Runtime overhead grows sharply with destination count; cost improvement must be weighed against this additional computation.
- Adaptive operator weighting has no consistent measured advantage in the supplied results. Bottleneck-aware destruction shows a lower cost in its ablation, but the capacity-invariant values limit the congestion-specific interpretation.

These experiments do not prove global optimality, universal superiority, monotonic improvement with destination count, or a capacity-dependent bottleneck effect. No formal approximation ratio is established.

7. Limitations and next steps

- Stage 2 uses shortest-path-based multicast routing rather than the original paper's complete routing optimization formulation.
- The evaluation uses one network topology and generated requests; other topologies are needed to assess generalization.
- Runtime scales poorly with destination count.
- Paired tests do not reach the 0.05 significance threshold for the reported destination experiments.
- The bottleneck ablation should be rechecked with per-seed outputs because cost is invariant across tested capacities.
- Adaptive weighting showed no consistent added value and should remain optional.

Next steps should prioritize independent-topology validation, stronger multicast-tree routing, per-seed congestion analysis, and faster local rerouting. Additional objectives such as delay, jitter, reliability, or energy should be added only after the core comparison is stable.

8. Conclusion

MA-ALNS is a multicast-aware neighborhood-search heuristic that starts from an LP-based sequential solution and then revisits VNF placements using structure-aware destroy/repair operators. The latest experiments show lower mean costs on some larger multicast instances, primarily through VNF consolidation. At 40 destinations, mean total cost falls by 11.72% and mean VNF count by 14.71%. However, gains are instance-dependent, no reported destination-level paired test reaches statistical significance at $\alpha = 0.05$, and runtime reaches about 6.75 seconds—roughly 80 times the baseline in the aggregate table. MA-ALNS is therefore a promising offline improvement heuristic, not a universally superior replacement for the faster sequential heuristic.

References

[1] M. Asgarian, G. Mirjalily, and Z.-Q. Luo, "Embedding Multicast Service Function Chains in NFV-Enabled Networks," IEEE Communications Letters, vol. 25, no. 4, pp. 1264–1268, Apr. 2021, doi: 10.1109/LCOMM.2020.3044894.

[2] B. Ren, D. Guo, Y. Shen, G. Tang, and X. Lin, "Embedding Service Function Tree With Minimum Cost for NFV-Enabled Multicast," IEEE Journal on Selected Areas in Communications, vol. 37, no. 5, pp. 1085–1097, May 2019.

[3] S. Q. Zhang, A. Tizghadam, B. Park, H. Bannazadeh, and A. Leon-Garcia, "Joint VNF Placement and Routing for Multicast Service on SDN," in Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS), 2016, pp. 333–341.

[4] G. Mirjalily and Z.-Q. Luo, "Optimal Network Function Virtualization and Service Function Chaining: A Survey," Chinese Journal of Electronics, vol. 27, no. 4, pp. 704–717, Jul. 2018.

[5] S. Ropke and D. Pisinger, "An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows," Transportation Science, vol. 40, no. 4, pp. 455–472, 2006.